



Basic AI for Everyone

Prof. Kenneth K.M. Lam

Centre for Advances in Reliability and Safety
(www.cairs.hk)

Department of EEE, HK PolyU

Email: enkmlam@polyu.edu.hk



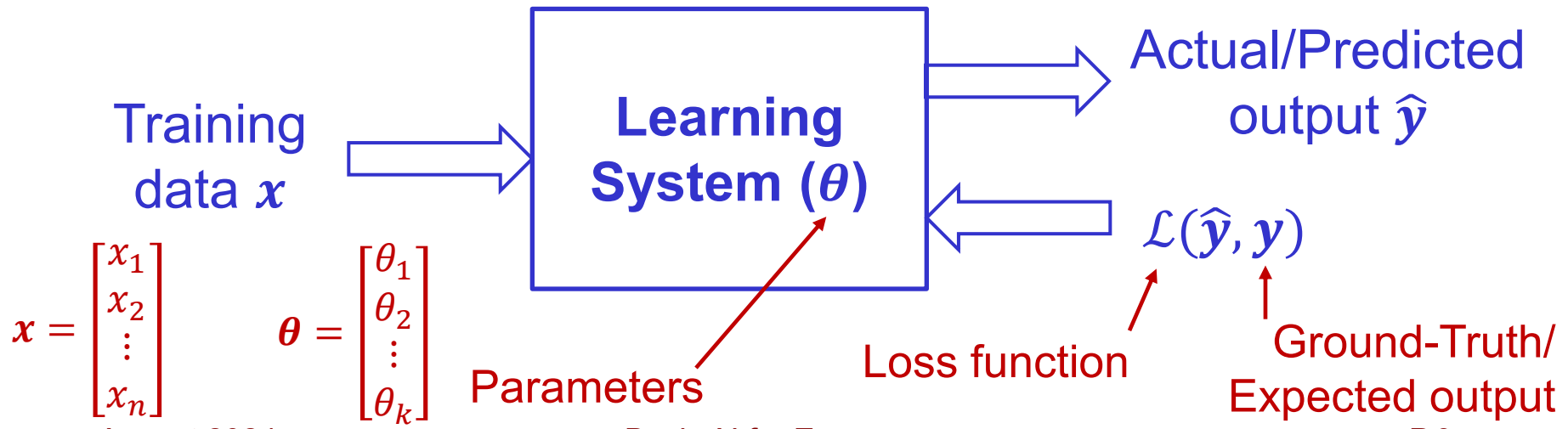
Part A (55 minutes)

- ☐ Learning Systems
- ☐ Learning: Supervised and Unsupervised
- ☐ Classification and Detection: Features
- ☐ Neuron and Perceptron
- ☐ Correlation/Dot Product
- ☐ Perceptron Algorithm
- ☐ Multilayer Neural Network
- ☐ Deep Neural Network:
Convolutional Neural Network (CNN)



Learning Systems

- The learning of a machine is based on three parts:
 - 1) The **data** to be learned by a method
 - 2) A **metric** that quantifies the error between the output and the idea output
 - 3) A **feedback** mechanism that uses the quantified error to guide the method to produce better results (backpropagation algorithm)





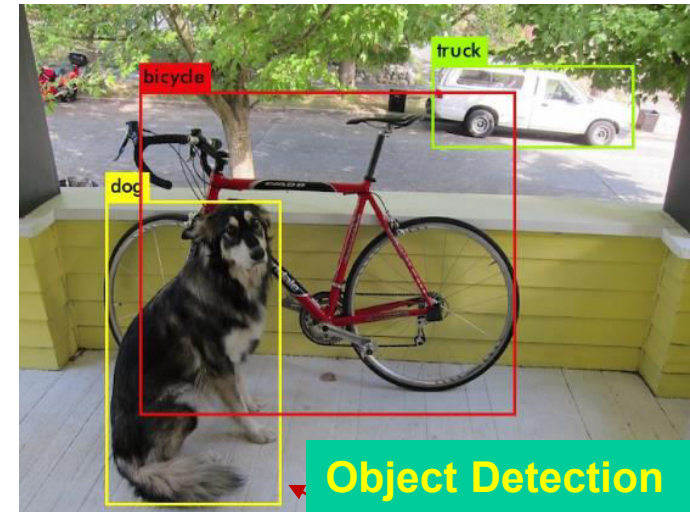
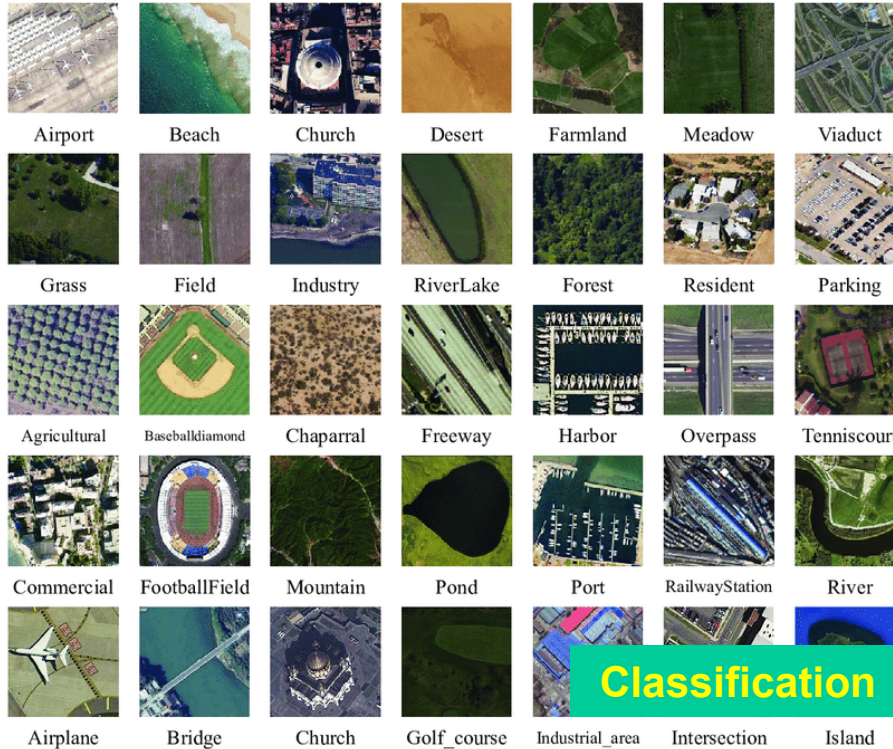
Learning: Supervised and Unsupervised

Types of Learning

- ❑ Machine Learning (ML) algorithms are broadly classified into three types:
 1. Supervised learning
 2. Unsupervised learning
 3. Reinforcement learning
- ❑ **Supervised Learning**
 - The data used for is labeled, i.e., the data has been divided into different classes
 - Examples: algorithms for image classification, object detection, face recognition, etc.



Classification and Detection: *CARS* Features



Bounding box

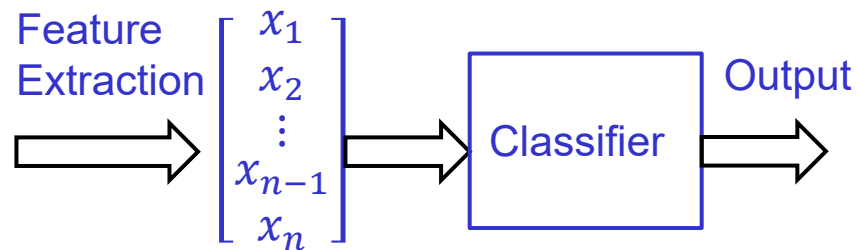
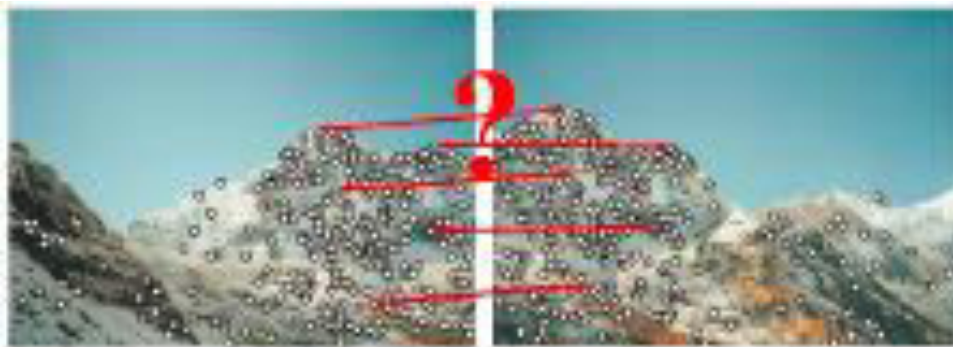
- Image annotation or labeling is a tedious and labor-intensive process
- For training an algorithm, hundreds of thousands of training samples are needed



Classification and Detection: *CARS* Features

Image Classification and Object Detection

- ❑ To perform classification and detection on an image, the pixel values will not be used directly
- ❑ **Features** (also called descriptors, meta-data, etc.) are extracted from the image
- ❑ Based on the features, classification or detection is performed by a **classifier**



an n -D
feature vector

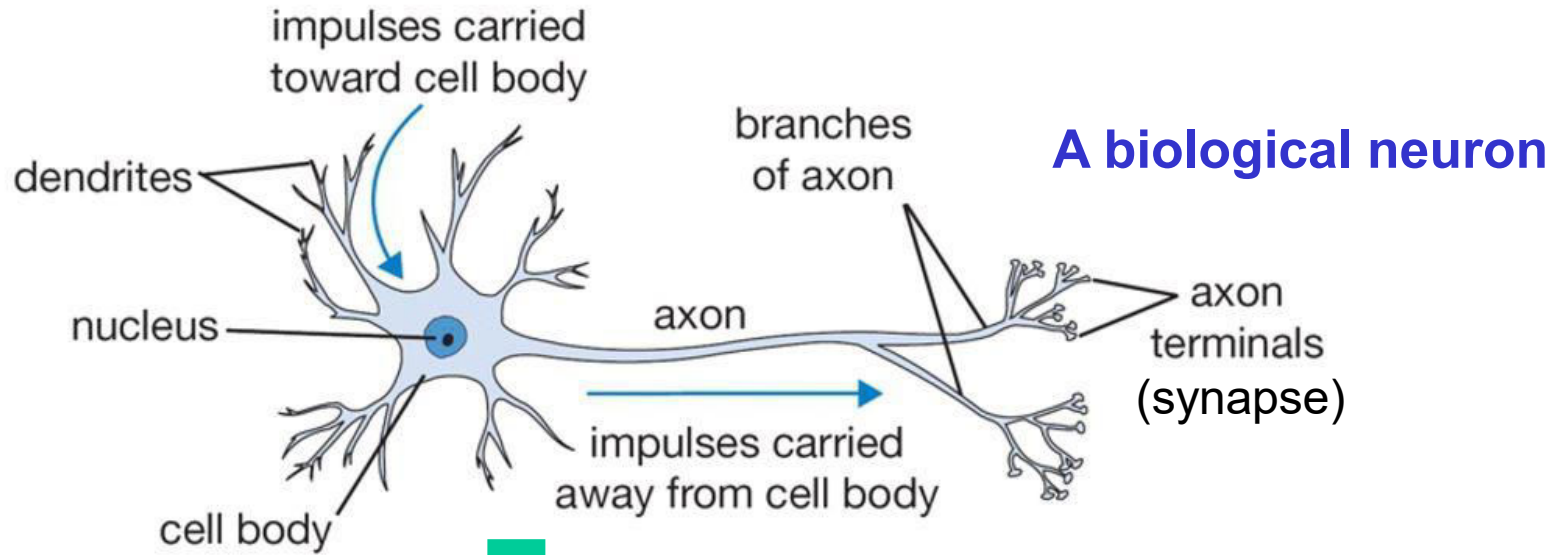


Classification and Detection: *CARS* Features

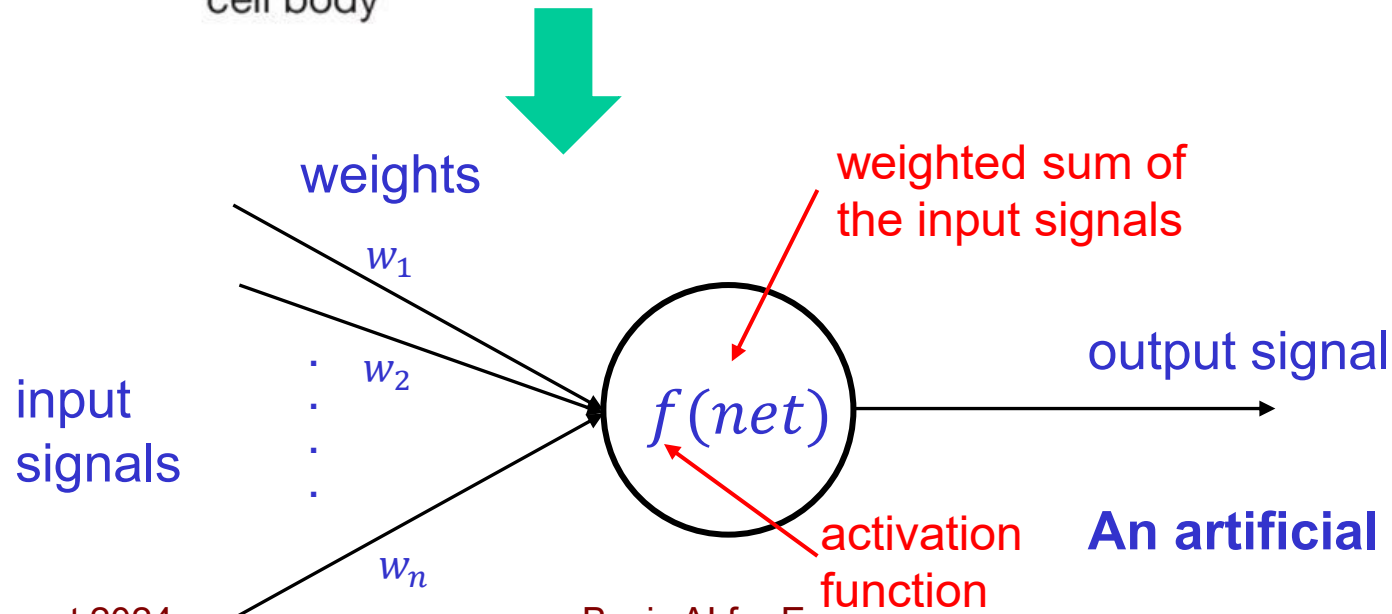
- ❑ Two types of features:
 - **Handcrafted features:** designed and created by humans based on their knowledge and experience
Examples: histogram of oriented gradients (HOG), local binary pattern (LBP), scale-invariant feature transform (SIFT), Gabor wavelets,
 - **Deep features:** learned by using a deep neural network with a huge amount of training samples
- ❑ The features need to be robust to or invariant to translation, scale, orientation, lighting conditions, etc.
- ❑ Classifiers: *k*-nearest neighbor (KNN), Support Vector Machine (SVM), Random Forest, ...



Neuron and Perceptron



A biological neuron



An artificial neuron

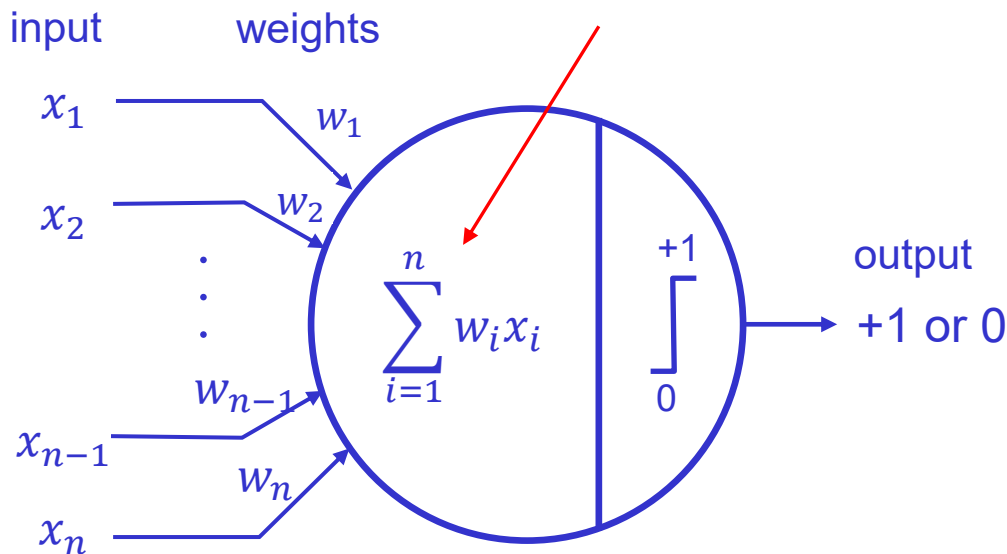


Neuron and Perceptron

□ Early 1940s, **McCulloch and Pitts**

- Proposed neuron models in the form of binary thresholding devices
- Can perform the basic logic operations (NOT, OR and AND)

$$S = w_1x_1 + w_2x_2 + \cdots + w_nx_n = [w_1 \quad \cdots \quad w_n] \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$



$$= \mathbf{w}^T \mathbf{x}$$

$x_i = 0 \text{ or } 1$
 $w_i = 1 \text{ or } -1$

a threshold

$$O^{k+1} = \begin{cases} 1 & \text{if } \sum_{i=1}^n w_i x_i^k \geq T \\ 0 & \text{otherwise} \end{cases}$$



Correlation/Dot Product

Notations of vectors

- ❑ A single value or variable is a **scalar**, denoted as x , y , w , z , etc.
- ❑ A **vector** is a quantity that contains more than one element (can be used to represent magnitude and direction), denoted as x , y , w , z , etc.
- ❑ Example:

$$S = w_1x_1 + w_2x_2 + \cdots + w_nx_n = [w_1 \quad \cdots \quad w_n] \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$$

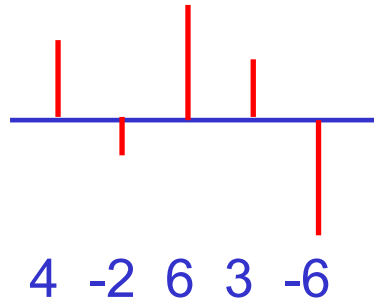
A column vector

A row vector

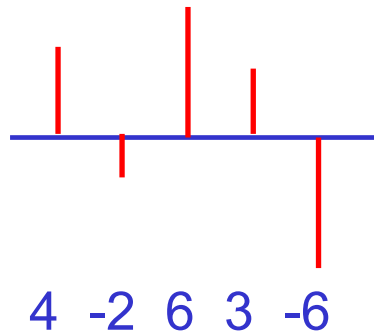
$$\text{where } \mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_n \end{bmatrix}, \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \mathbf{w}^T \mathbf{x} \text{ or } \vec{w}^T \vec{x}$$



Dot product / Correlation



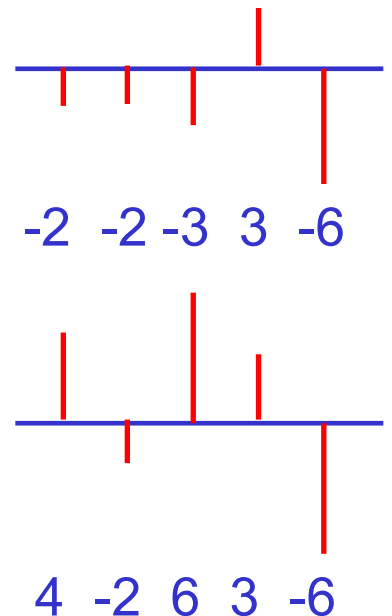
$$\begin{aligned} &= 4 \times 4 + (-2) \times (-2) + 6 \times 6 + 3 \times 3 \\ &\quad + (-6) \times (-6) \\ &= 101 \end{aligned}$$



The two sequences are the same

$$\begin{aligned} &= (-2) \times 4 + (-2) \times (-2) + (-3) \times 6 \\ &\quad + 3 \times 3 + (-6) \times (-6) \\ &= 23 \end{aligned}$$

The two sequences are not the same





Correlation/Dot Product

–The dot-product operation is to measure the similarity between two functions or sequences

$$\int x(t) e^{-j2\pi k f_0 t} dt \approx \sum_p \underbrace{(x(p\Delta t)\Delta t)}_{\text{signal}} \cdot \underbrace{e^{-j2\pi k f_0 p\Delta t}}_{\text{kernel or basis function}}$$

$\cos(2\pi k f_0 t) + j \sin(2\pi k f_0 t)$

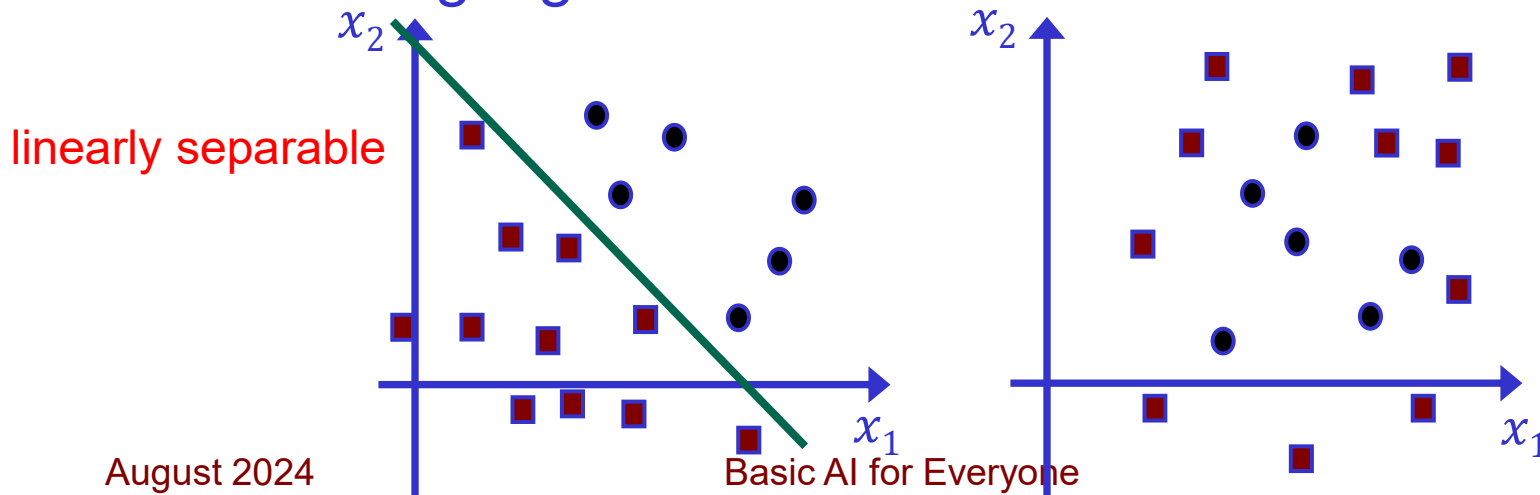
$$= x_1 e_1 + x_2 e_2 + \cdots + x_n e_n$$

$$X[k] = \frac{1}{T_0} \int_{-T_0/2}^{T_0/2} x(t) e^{-j2\pi k f_0 t} dt \Rightarrow \text{Measure the frequency content of frequency } k f_0 \text{ Hz in the signal } x(t)$$



Perceptron Algorithm

- Mid-1950s – early 1960s, **Rosenblatt**
 - **Perceptrons**, when trained with **linearly separable training sets**, would converge to a solution in a finite number of iterative steps
 - The basic perceptron networks were inadequate for most pattern recognition tasks
 - Multiple layers of perceptrons – lacked effective training algorithms





Perceptron Algorithm

x is a n -D vector

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}, \mathbf{w}_a = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \\ b \end{bmatrix}$$

- A single **perceptron** unit learns a linear boundary between two **linearly separable** pattern classes

- Given $x \in \mathbb{R}^n$, a linear decision function : bias, bias coefficient, bias weight

$$\begin{aligned} d(x, \mathbf{w}_a) &= w_1x_1 + w_2x_2 + \cdots + w_nx_n + b \\ &= \mathbf{w}^T \mathbf{x} + b = \mathbf{w}_a^T \mathbf{x}_a \end{aligned}$$

Augmented weight vector dot product

where $\mathbf{w}_a = [w_1, w_2, \dots, b]^T$, $\mathbf{x}_a = [x_1, x_2, \dots, x_n, 1]^T$

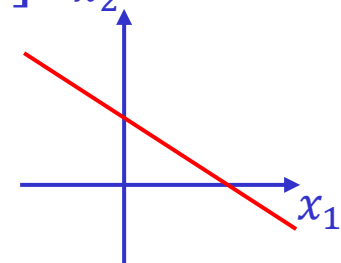
- Consider a two-class problem with samples

$$\mathbf{x}^{1,j}, \quad 1 \leq j \leq M_1 \text{ in Class 1}$$

$$\mathbf{x}^{2,j}, \quad 1 \leq j \leq M_2 \text{ in Class 2}$$

$$\text{suppose } c(\mathbf{x}) = 1 \text{ or } (+1) \quad \text{if } d(\mathbf{x}_a, \mathbf{w}_a) > 0$$

$$c(\mathbf{x}) = 2 \text{ or } (-1) \quad \text{if } d(\mathbf{x}_a, \mathbf{w}_a) < 0$$

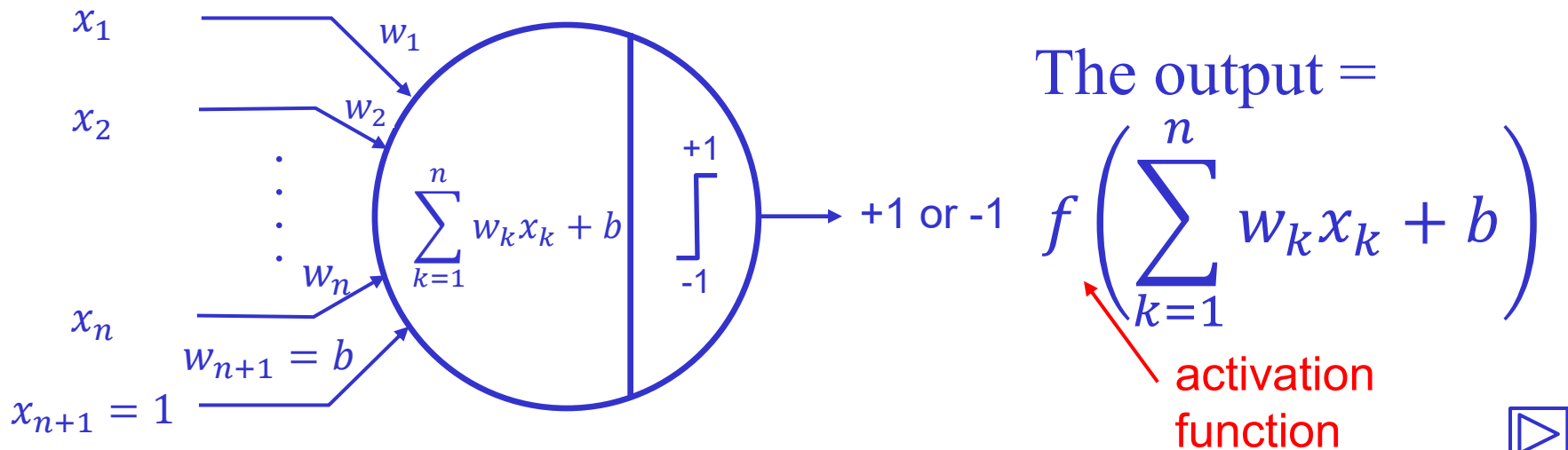


$$w_1x_1 + w_2x_2 + b = 0$$

line
plane
hyperplane



Perceptron Algorithm



Slide 22

- ❑ The activation function is a thresholding function
- ❑ The perceptron attempts to find a solution by iteratively stepping through the patterns of each of two classes
- ❑ Starting with an arbitrary weight vector and bias, the perceptron is guaranteed to converge in a finite number of iterations if the classes are linearly separable



Perceptron Algorithm

- The M_1 samples of Class 1 and M_2 samples of Class 2 are used to learn the weights

$$\left. \begin{array}{l} w_1 x_1^{1,1} + w_2 x_2^{1,1} + \cdots + w_n x_n^{1,1} + b > 0 \\ w_1 x_1^{1,2} + w_2 x_2^{1,2} + \cdots + w_n x_n^{1,2} + b > 0 \\ \vdots \\ w_1 x_1^{1,M_1} + w_2 x_2^{1,M_1} + \cdots + w_n x_n^{1,M_1} + b > 0 \end{array} \right\}$$
$$\left. \begin{array}{l} w_1 x_1^{2,1} + w_2 x_2^{2,1} + \cdots + w_n x_n^{2,1} + b < 0 \\ w_1 x_1^{2,2} + w_2 x_2^{2,2} + \cdots + w_n x_n^{2,2} + b < 0 \\ \vdots \\ w_1 x_1^{2,M_2} + w_2 x_2^{2,M_2} + \cdots + w_n x_n^{2,M_2} + b < 0 \end{array} \right\}$$



Perceptron Algorithm

- For a fixed set of samples, w and b can be determined by finding a solution of the inequalities

$$\mathbf{w}_a^T \mathbf{x}_a \rightarrow \left. \begin{aligned} w_1 x_1^{1,i} + \dots + w_n x_n^{1,i} + b &> 0 & 1 \leq i \leq M_1 \\ -w_1 x_1^{2,j} - \dots - w_n x_n^{2,j} - b &> 0 & 1 \leq i \leq M_2 \end{aligned} \right\}$$

- Determine the linear boundaries by the use of a variety of algorithms, which recursively *train* the weight vectors until the inequalities are satisfied

$$\mathbf{w}_a = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \\ b \end{bmatrix}$$



Perceptron Algorithm

$$(\mathbf{w}_a + \alpha \mathbf{x}_a^{1,i})^T \mathbf{x}_a^{1,i} = \underbrace{\mathbf{w}_a^T \mathbf{x}_a^{1,i}}_{+ve} + \alpha (\mathbf{x}_a^{1,i})^T \mathbf{x}_a^{1,i}$$

Step 1: Choose $\mathbf{w}_a$ arbitrarily

Step 2: Evaluate $p_1 = \mathbf{w}_a^T \mathbf{x}_a^{1,i}$ (a sample from Class 1)

if $p_1 \leq 0$ (i.e. misclassification)

$$\begin{bmatrix} a & b \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = a^2 + b^2$$

$$\mathbf{w}_a \leftarrow \mathbf{w}_a + \alpha \mathbf{x}_a^{1,i}$$

otherwise leave $\mathbf{w}_a$ unchanged

Step 3: Evaluate $q_1 = \mathbf{w}_a^T \mathbf{x}_a^{2,j}$ (a sample from Class 2)

if $q_1 \geq 0$ (i.e. misclassification)

$$\mathbf{w}_a \leftarrow \mathbf{w}_a - \alpha \mathbf{x}_a^{2,j}$$

α : correction increment,
learning rate

otherwise leave $\mathbf{w}_a$ unchanged

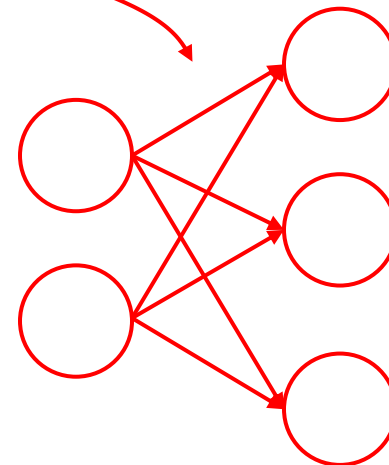
Repeat Steps 2 and 3 until convergence



Multilayer Neural Network

- 1986, **Rumelhart, Hinton, and Williams**
 - Developed new training algorithms for multilayers of perceptron-like units
 - **Backpropagation** (*backprop*): an effective training method for multilayer networks, as well as for deep neural networks
 - Multilayer, fully connected neural networks, whose *inputs are pattern vectors*

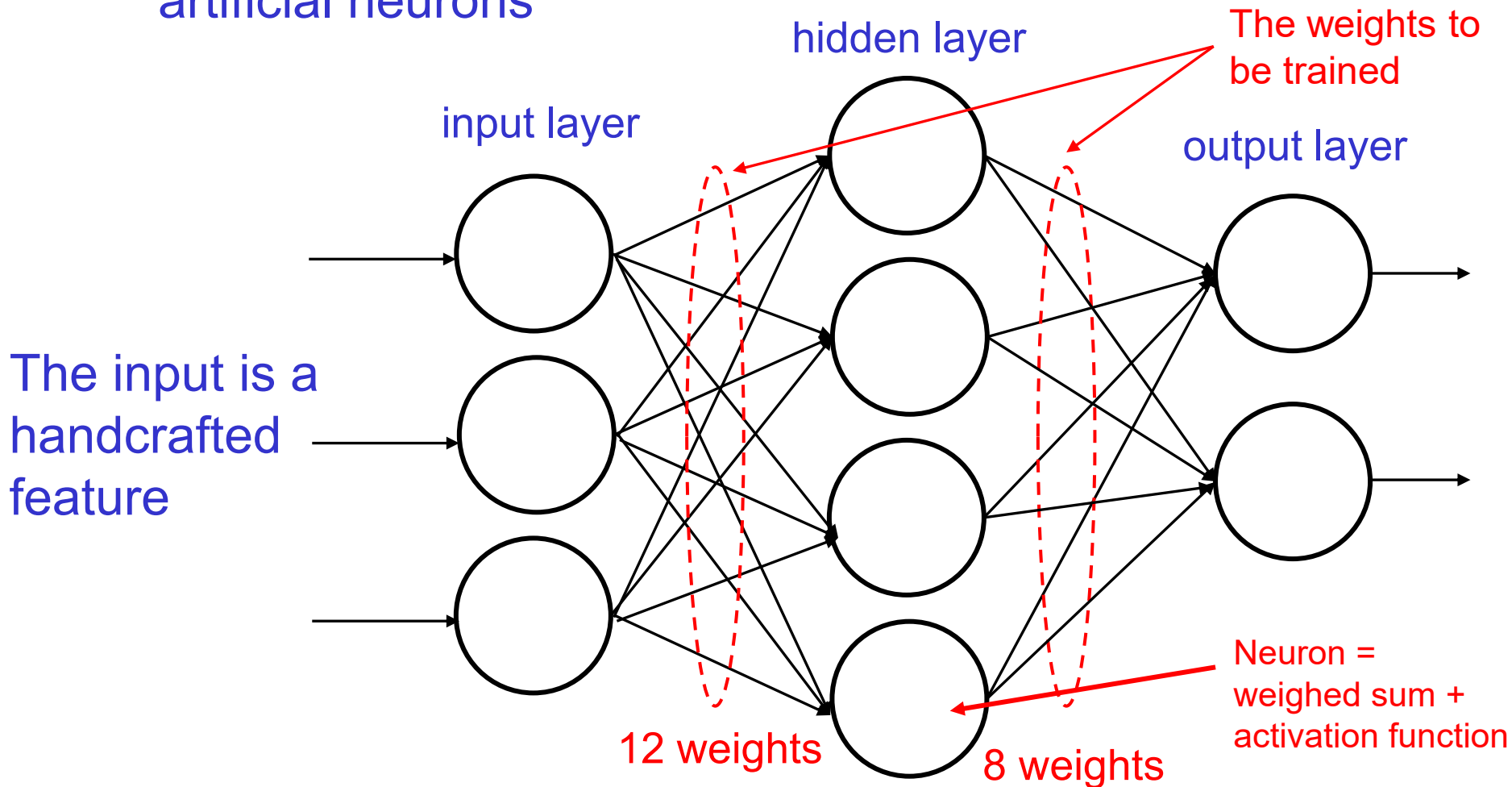
Based on human-engineering techniques, i.e., handcrafted features





Multilayer Neural Network

- An artificial neural network is a layered network of artificial neurons





Deep Neural Network

- ❑ Deep learning is an extension of the neural network
- ❑ A **deep neural network** is a multi-layer neural network that contains two or more hidden layers
- ❑ The backpropagation algorithm experiences three difficulties in the training process of the deep neural network:
 1. Vanishing gradient
 2. Overfitting
 3. Computational load
- ❑ To solve the vanishing gradient problem, the **Rectified Linear Unit (ReLU)** function is used as the activation function



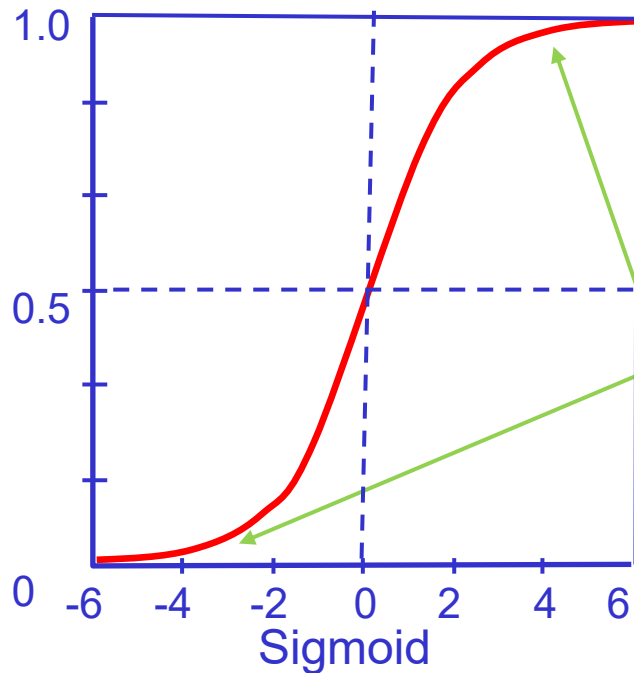
Deep Neural Network

- For multilayer neural network, the activation function used is the sigmoid function

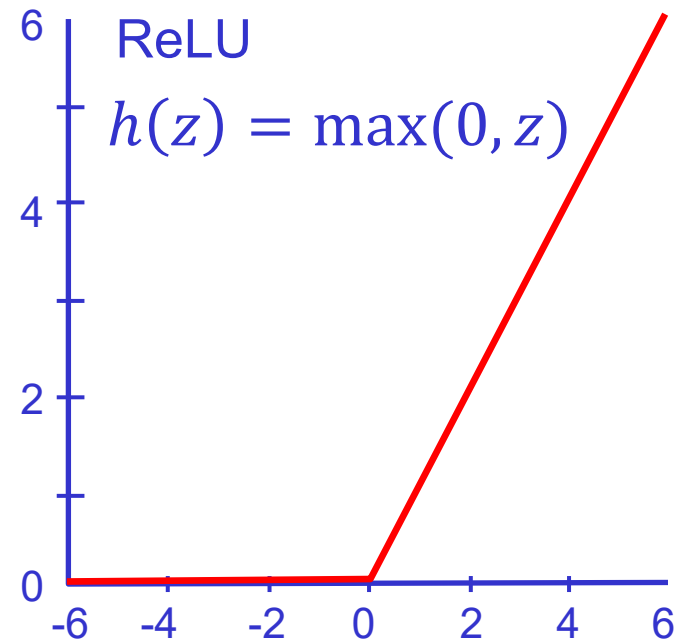
$$h(z) = \frac{1}{1 + e^{-z}} \quad \text{sigmoid function}$$



Slide 15



Gradients
close to
zero





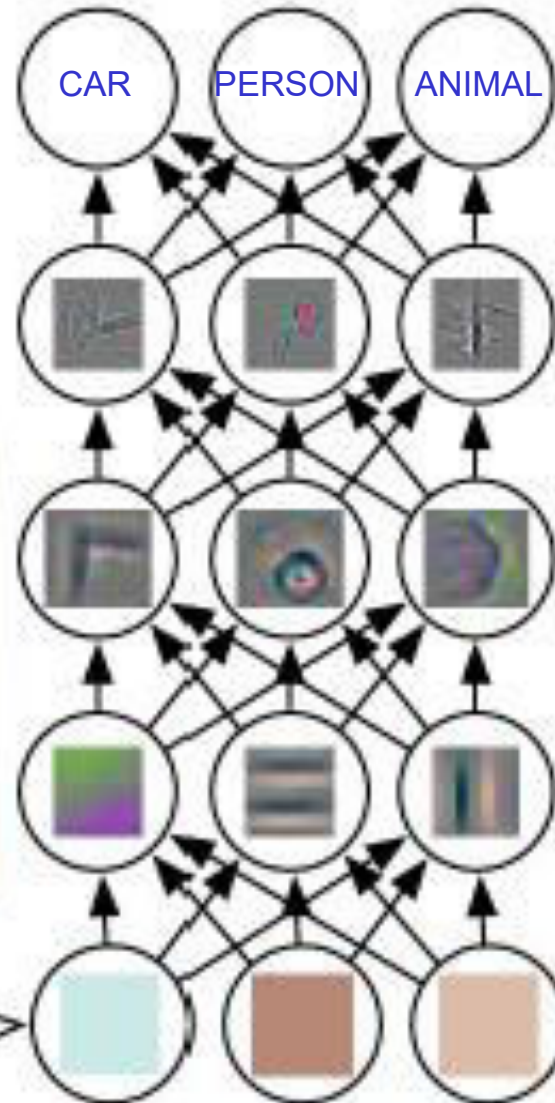
Deep Neural Network

- ❑ Overfitting: the trained network achieved very good performance on training data, but poor performance on test data
Note that the training data set and test data set do not overlap
- ❑ Various methods have been proposed to solve the overfitting problem (dropout, data augmentation, ...)
- ❑ Computational load is relieved to a considerable extent by the use of **mini-batch** and the introduction of high-performance hardware, such as **Graphics Processing Unit (GPU)**



Deep Neural Network

Feature extraction and classification are learned simultaneously



Output
(object identity)

3rd hidden layer
(object parts)

2nd hidden layer
(corners and
contours)

1st hidden layer
(edges)

Visible layer
(input pixels)



Deep Neural Network

□ Overfitting

- When training a machine learning model, the training error (e.g. $\text{MSE}_{\text{train}}$) is minimized
- The **generalization error**, also called the **test error** (e.g. MSE_{test}), is to be low as well
 - Estimated by measuring the performance on a test set of examples that are collected separately from the training set
- The factors determining how well a learning algorithm will perform are its ability to
 1. make the training error small
 2. make the gap between training and test errors small

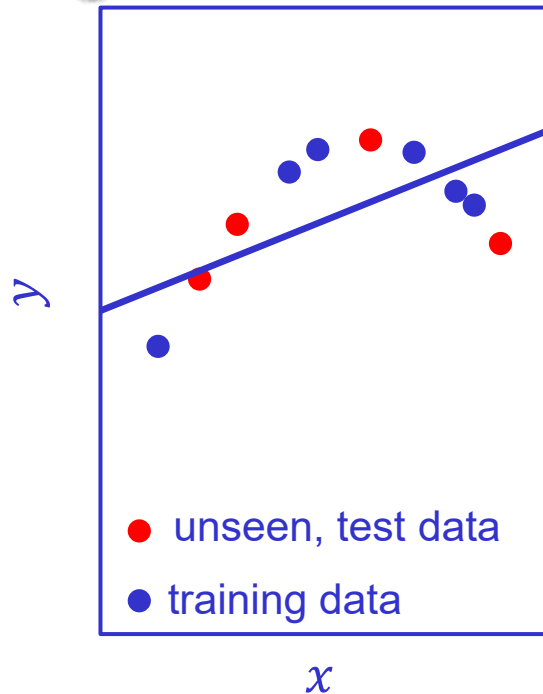


Deep Neural Network

- ❑ **Underfitting** occurs when the model is not able to obtain a sufficiently low error rate on the training set
- ❑ **Overfitting** occurs when the gap between the training and test errors is too large
- ❑ A model can be controlled to be more likely to overfit or underfit by altering its **capacity**
- ❑ To train deep neural networks, different techniques have been used to avoid overfitting



Underfitting

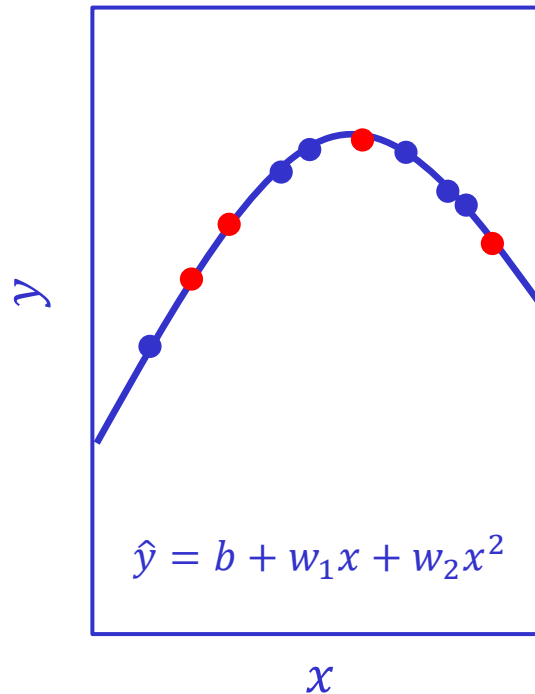


linear function

Large training error

Large test error

Appropriate capacity

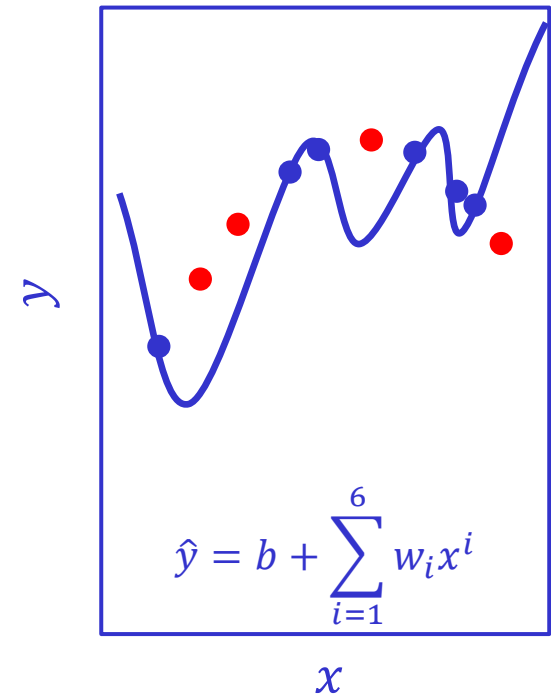


quadratic function

Small training error

Small test error

Overfitting



a polynomial of degree 6

Small training error

Large test error

Among the competing hypotheses or learning models that explain known observations equally well, the “simplest” one should be chosen



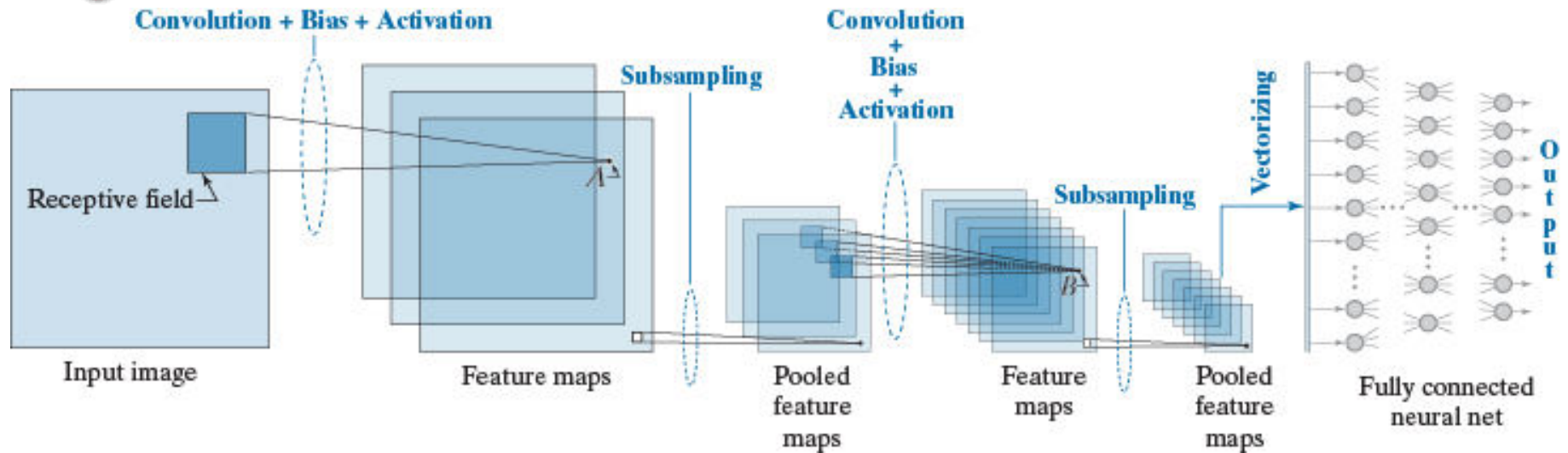
Deep Neural Network

Convolutional Neural Network (CNN)

- ❑ A class of deep, feed-forward artificial neural networks
- ❑ Training images are input directly into CNN, and the network learns the necessary features on its own
- ❑ Inputting images: the spatial relationships that may exist between pixels in an image can be retained and utilized
- ❑ Inspired by biological processes: the connectivity pattern between neurons resembles the organization of the animal visual cortex
 - Individual cortical neurons respond to stimuli only in a restricted region of the visual field (*receptive field*)



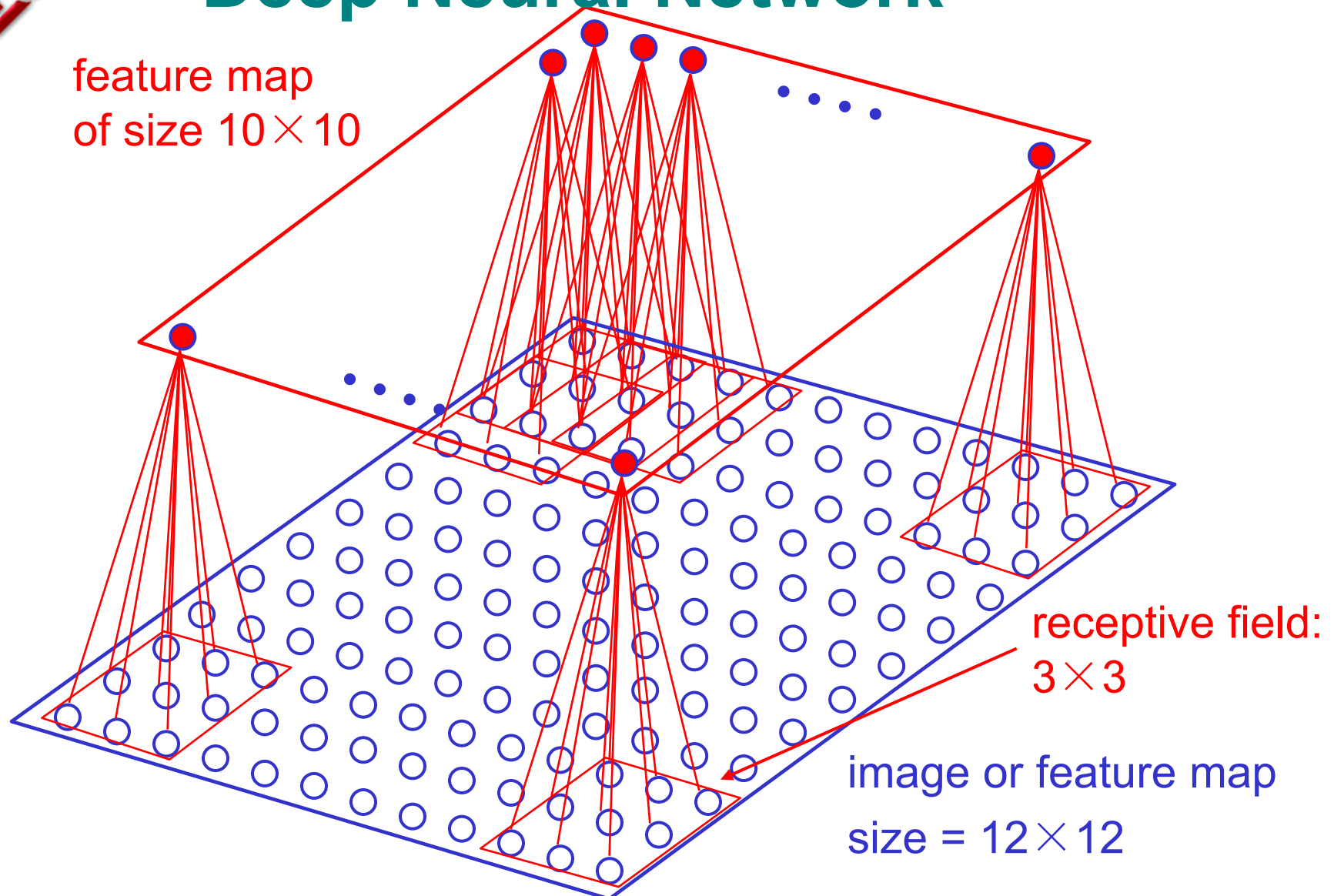
Deep Neural Network



- ❑ Every input of a layer is fed by a single value, determined by the convolution over a spatial neighborhood (receptive field) in the output of the previous layer
- ❑ Convolution computes a sum of products between pixels and a set of kernel weights
- ❑ Convolution + Bias $\rightarrow$ Activation



Deep Neural Network





Deep Neural Network

- A simple **handcrafted feature** for edge detection
 - Sobel operators (one for horizontal edges and the other for vertical edges)

$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$ $S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$



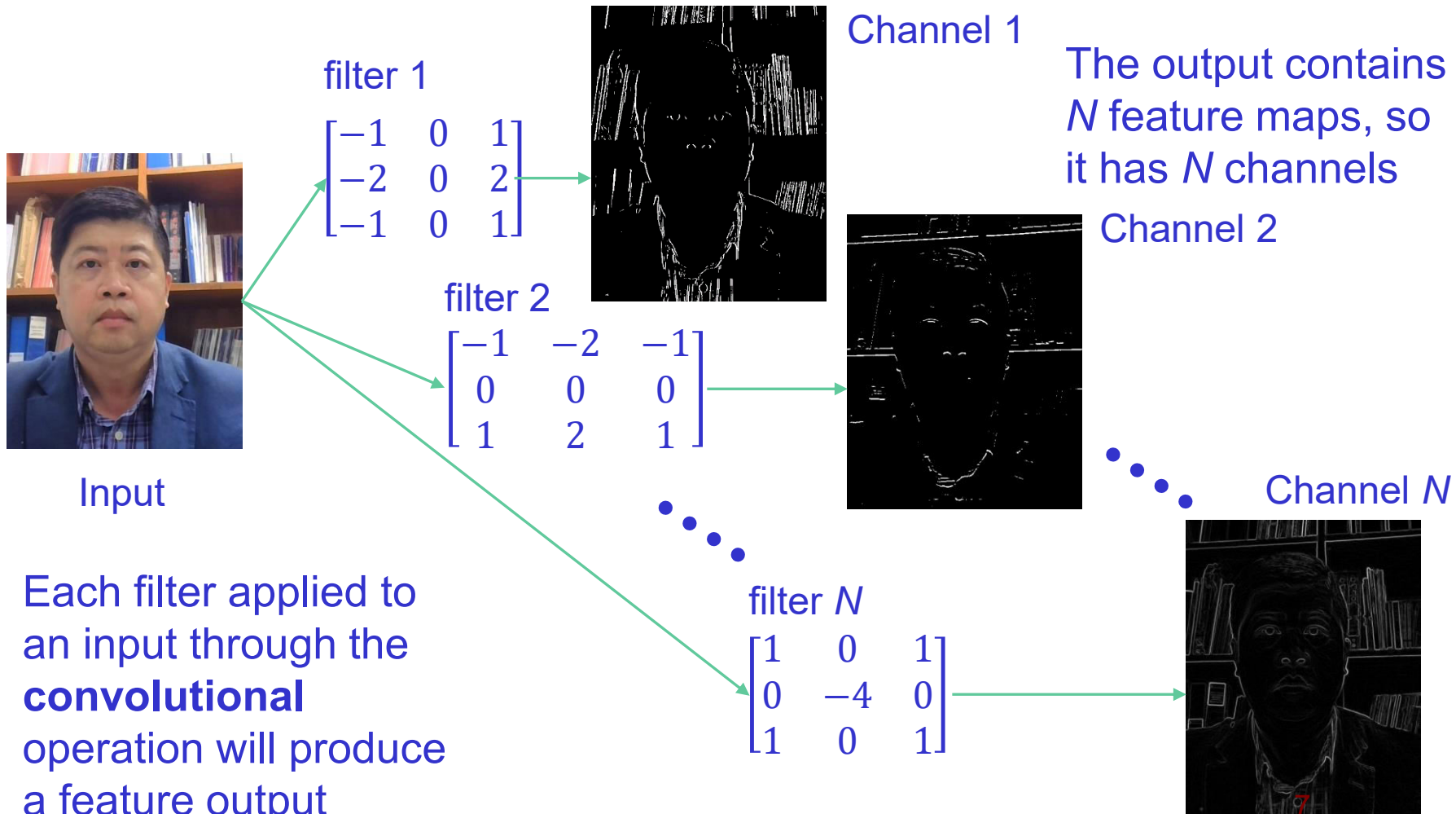
139	220	221	219
68	142	220	218
42	60	138	216
50	55	58	140

$$f_x = -1 \times 139 + 1 \times 221 - 2 \times 68 + 2 \times 220 - 1 \times 42 + 1 \times 138 = 482$$
$$f_y = -1 \times 139 + 1 \times 42 - 2 \times 220 + 2 \times 60 - 1 \times 221 + 1 \times 138 = -500$$
$$|\nabla f| = \sqrt{482^2 + 500^2} = 694.5$$
$$\angle \nabla f = \tan^{-1} \left(\frac{-500}{482} \right) = -46.05^\circ$$



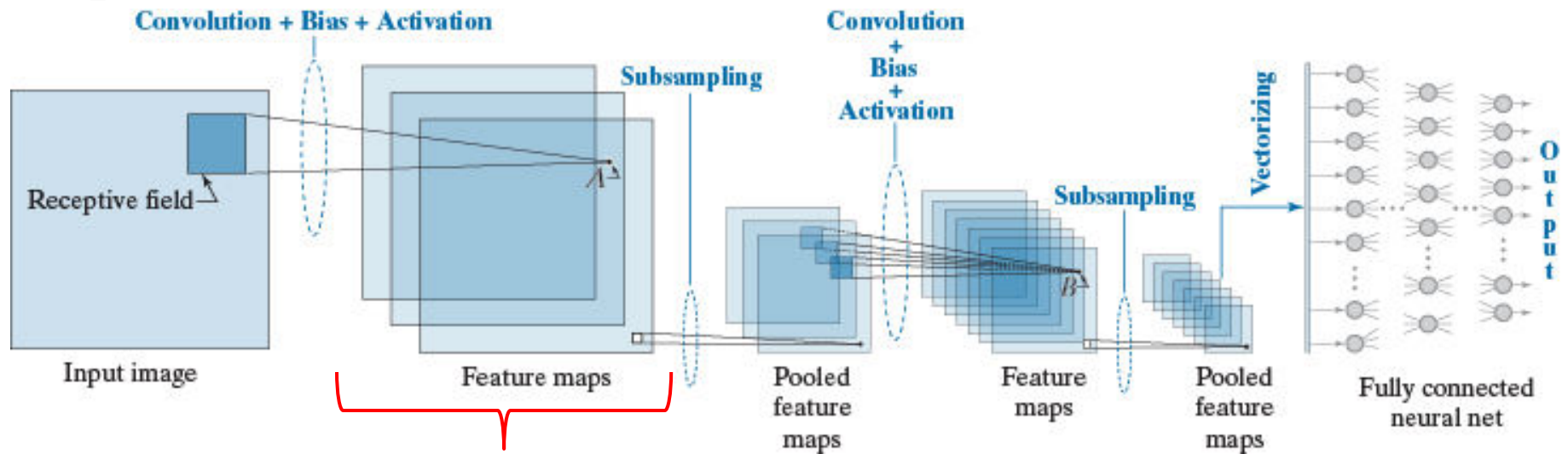
Deep Neural Network

Convolutional Neural Networks





Deep Neural Network

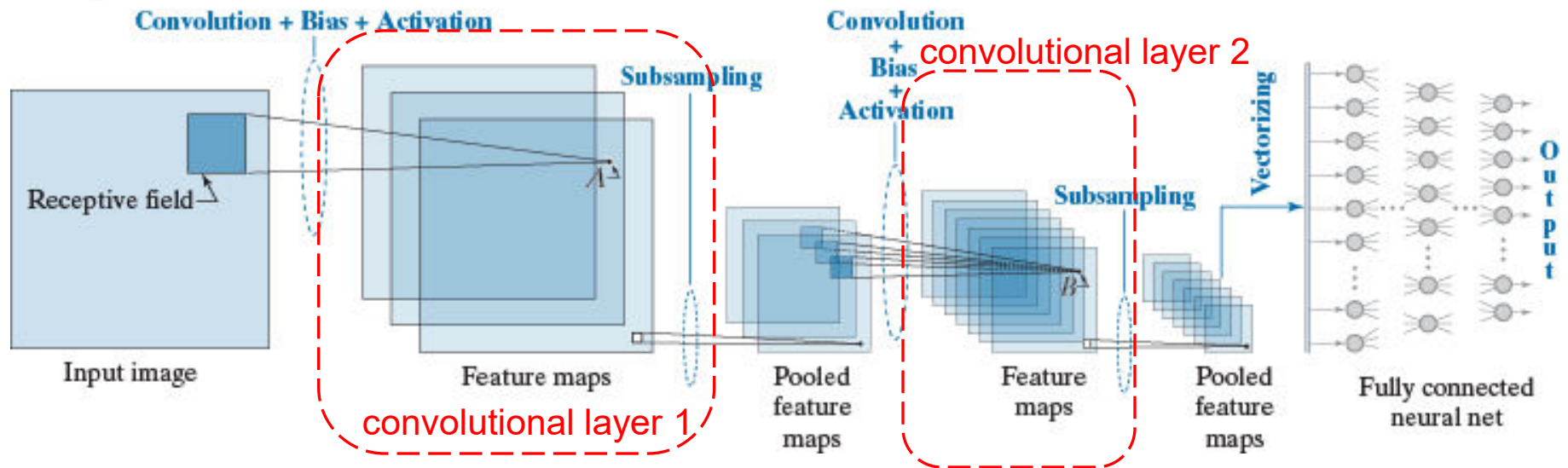


Three different kernels

- ❑ The **receptive field** is moved over the image and, at each location, convolution (a sum of products of the kernel weights and the pixels contained in the receptive field) is computed
- ❑ Each convolution value is added a bias, then passed through an activation function (ReLU)



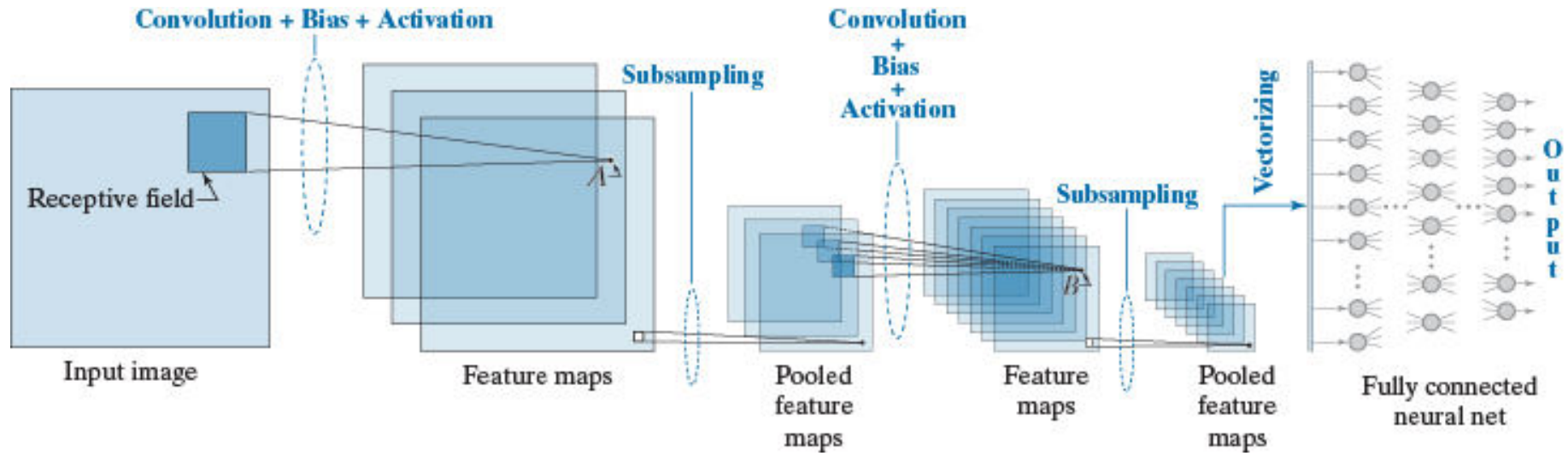
Deep Neural Network



- ❑ The result is stored in a **feature map** – convolution is to extract features such as edges, points, and blobs from the input
- ❑ A feature map is produced by using the same weights and a single bias (the kernel) – **weight (or parameter) sharing**
- ❑ The feature maps are referred to collectively as a **convolutional layer**



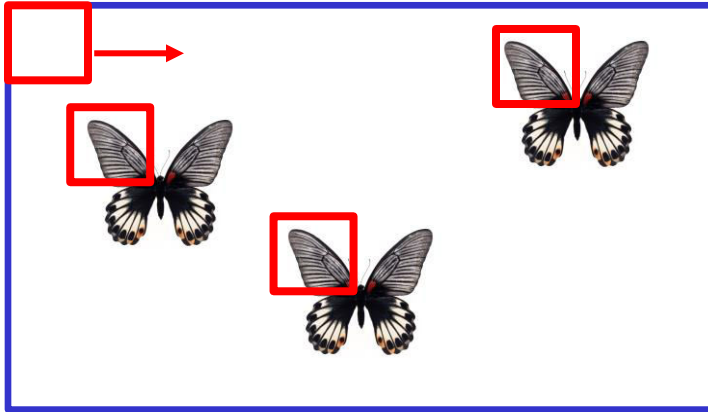
Deep Neural Network



- ❑ CNN is also called shift-invariant NN
- ❑ The process after convolution and activation is **subsampling**, also called **pooling**
- ❑ A reduction in spatial resolution can make the features more robust



Deep Neural Network



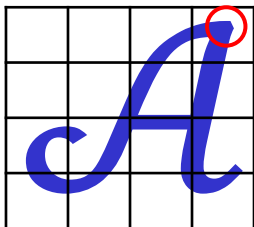
Translation Invariance:

The kernel is slid over the image, so the same object at different locations can be detected

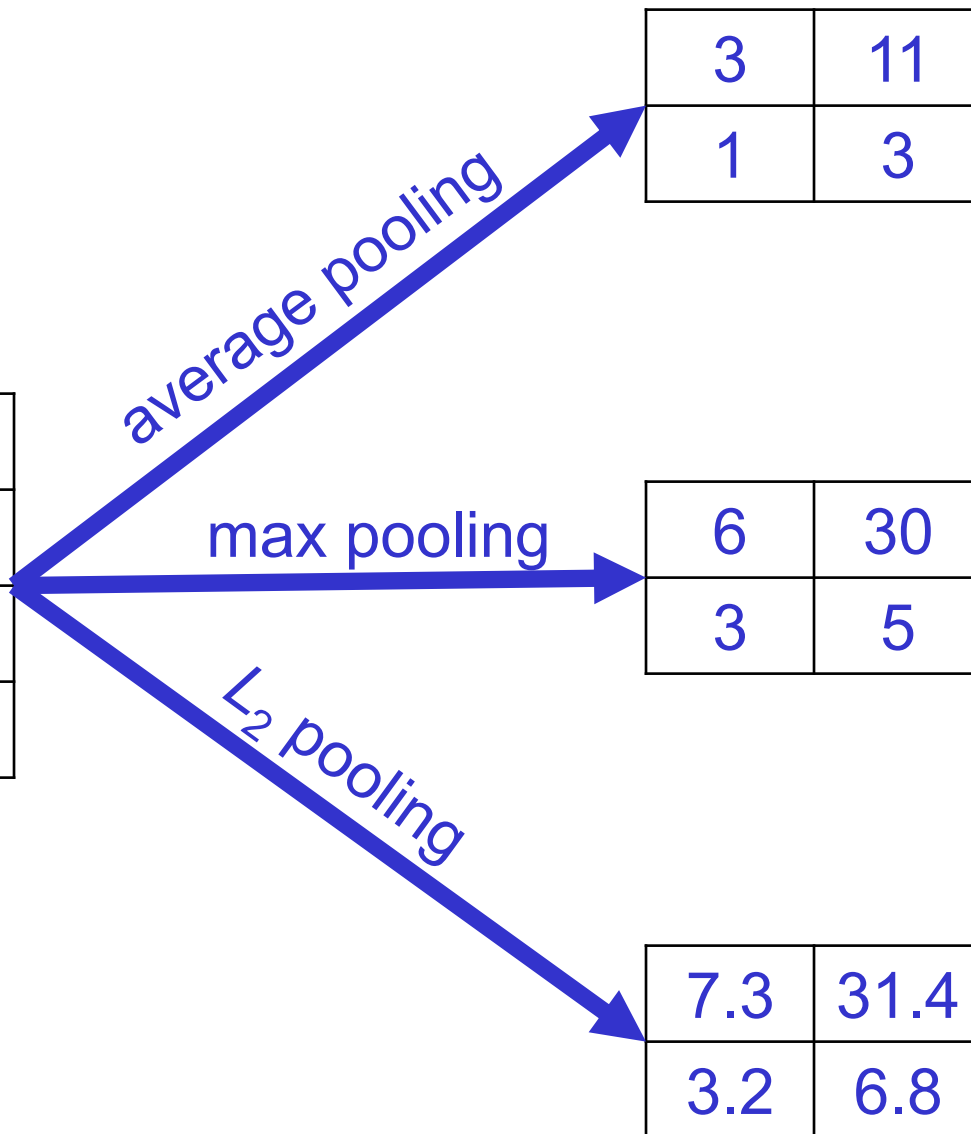
- ❑ Pooling can reduce the volume of data being processed
- ❑ Produced **pooled feature maps**
- ❑ Three common pooling methods:
 - Average pooling
 - Max-pooling
 - L_2 pooling



Introduction to DNN

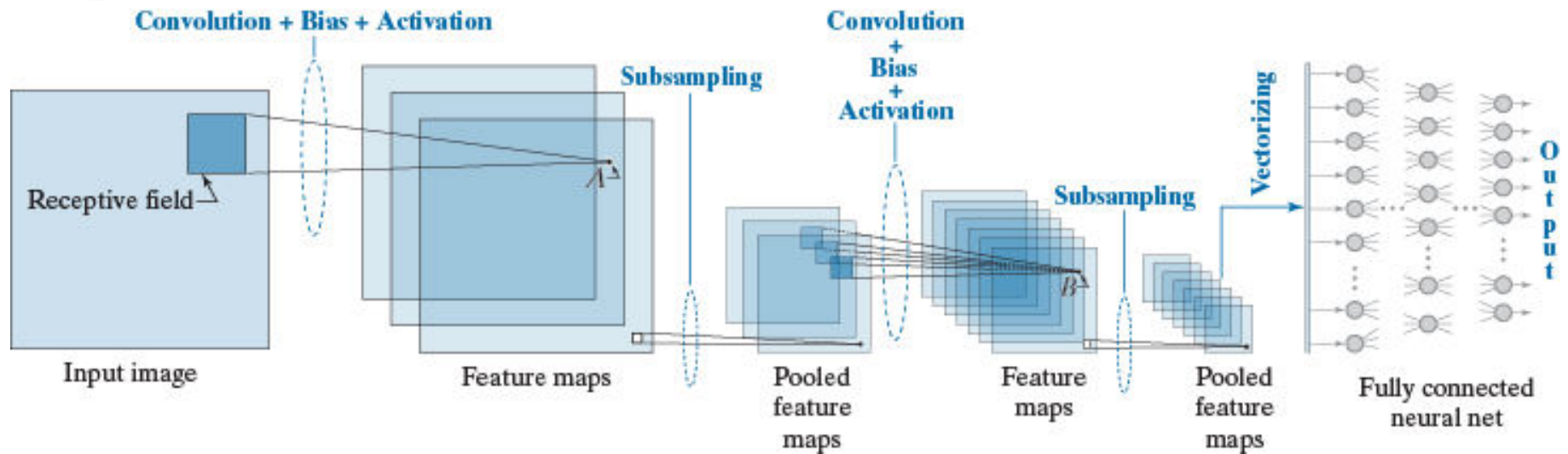


1	1	1	30
4	6	5	8
3	0	1	5
0	1	2	4





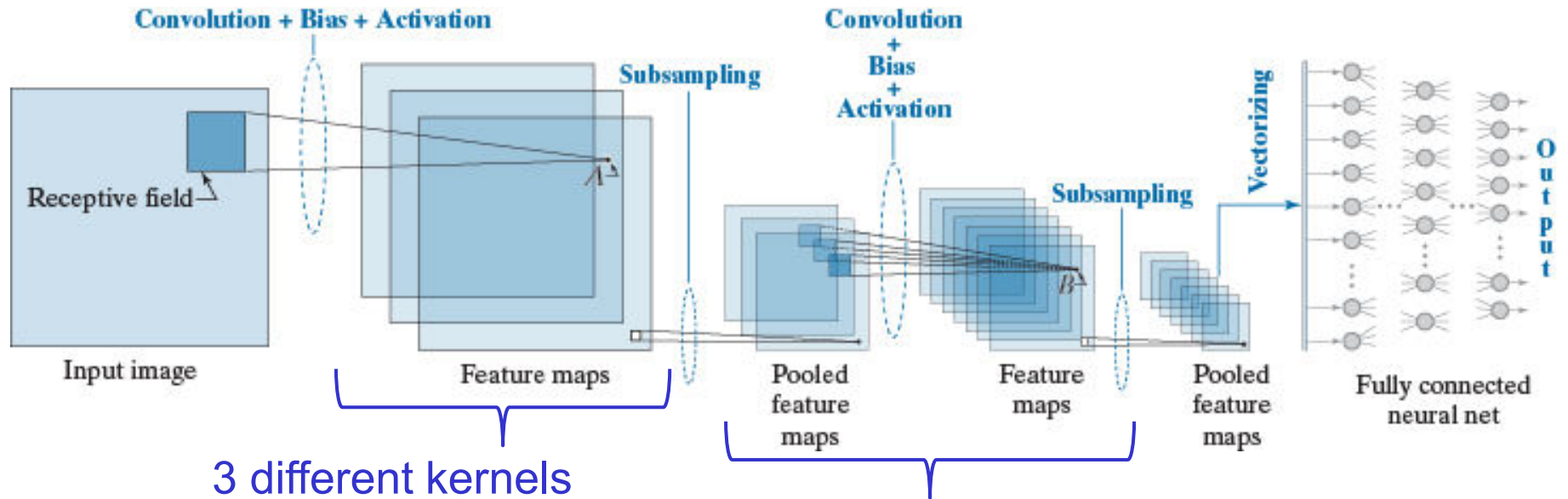
Deep Neural Network



- ❑ The pooled feature maps are referred to collectively as a **pooling layer**
- ❑ The second convolutional layer has the pooled feature maps as its multiple inputs
- ❑ For each feature map in the second layer, a different kernel (weights + bias) is applied to a pooled feature map in the previous layer to generate a value



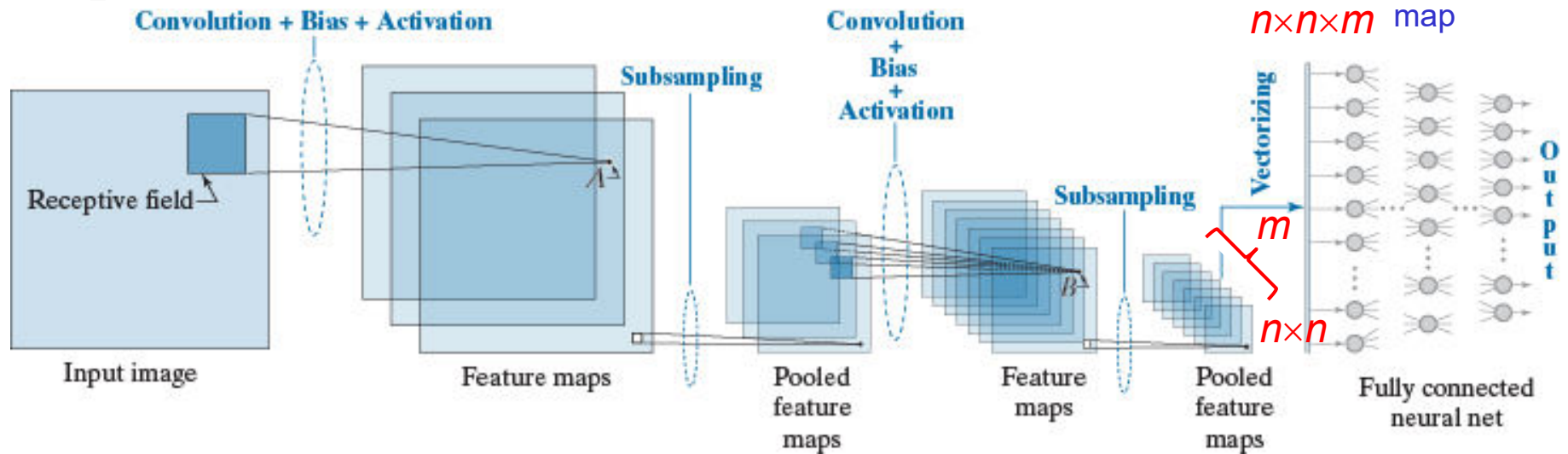
Deep Neural Network



- ❑ Convolution is a linear operation, so the respective values are added into one
- ❑ Number of kernels needed = 3 (1st layer) + 7 (2nd layer)
- ❑ For 5×5 receptive field, the number of parameters to be learned = $(5 \times 5 + 1) \times 3 + (5 \times 5 \times 3 + 1) \times 7 = 610$



Deep Neural Network



- ❑ The 2-D pooled feature maps in the last layer are vectorized to form a column vector, which is fed into a fully connected neural net
- ❑ The number of outputs in the fully connected net is equal to the number of pattern classes being classified
- ❑ The output with the highest value determines the class of the input



END & Thank you



Example 1

Perceptron Algorithm

Consider two pattern vectors,
of two classes, as follows:

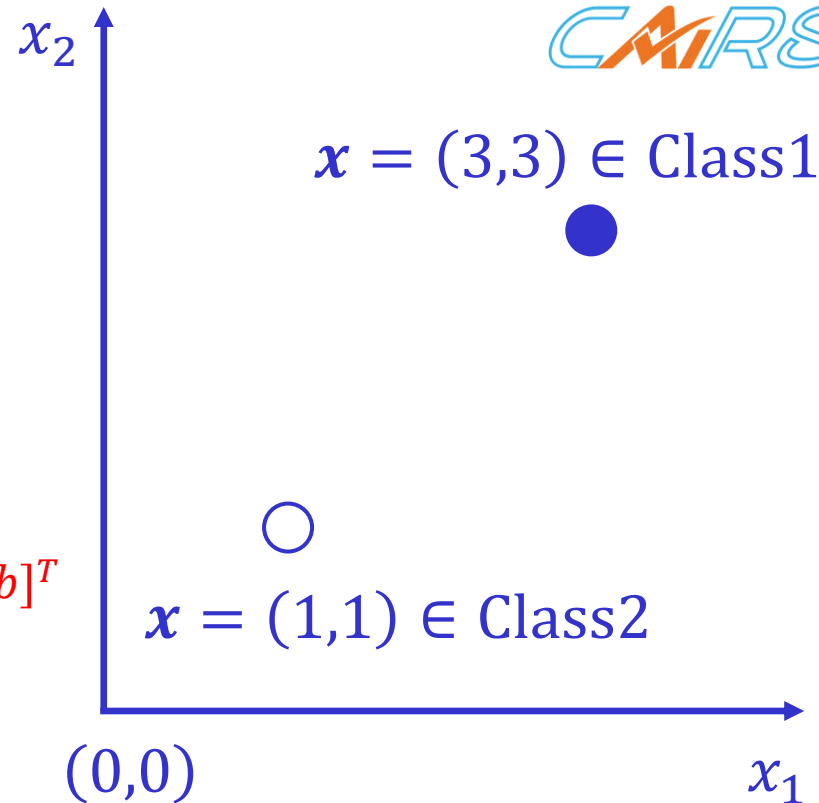
$$\mathbf{x} = [3 \quad 3 \quad 1]^T \text{ and}$$

$$\mathbf{x} = [1 \quad 1 \quad 1]^T \quad \mathbf{w} = [w_1 \quad w_2 \quad b]^T$$

The vectors are augmented
by appending a 1 at the end

The patterns are cycled through the perceptron, and one
complete iteration through all patterns of the training is
called an epoch

Set $\alpha = 1$ and initial weight vector $\mathbf{w}(1) = \mathbf{0} = [0 \quad 0 \quad 0]^T$





$$\square k = 1, \mathbf{x}(1) = [3 \quad 3 \quad 1]^T \in \omega_1, \mathbf{w}(1) = \mathbf{0} = [0 \quad 0 \quad 0]^T$$

$$\mathbf{w}^T(1)\mathbf{x}(1) = [0 \quad 0 \quad 0] \begin{bmatrix} 3 \\ 3 \\ 1 \end{bmatrix} = 0$$

$$\mathbf{w}(2) = \mathbf{w}(1) + \alpha \mathbf{x}(1) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + (1) \begin{bmatrix} 3 \\ 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \\ 1 \end{bmatrix}$$

$$\square k = 2, \mathbf{x}(2) = [1 \quad 1 \quad 1]^T \in \omega_2, \mathbf{w}(2) = [3 \quad 3 \quad 1]^T$$

$$\mathbf{w}^T(2)\mathbf{x}(2) = [3 \quad 3 \quad 1] \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = 7$$

$$\mathbf{w}(3) = \mathbf{w}(2) - \alpha \mathbf{x}(2) = \begin{bmatrix} 3 \\ 3 \\ 1 \end{bmatrix} - (1) \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \\ 0 \end{bmatrix}$$



□ A complete training epoch has been gone through

□ $k = 3, \mathbf{x}(3) = [3 \ 3 \ 1]^T \in \omega_1, \mathbf{w}(3) = [2 \ 2 \ 0]^T$

$$\mathbf{w}^T(3)\mathbf{x}(3) = [2 \ 2 \ 0] \begin{bmatrix} 3 \\ 3 \\ 1 \end{bmatrix} = 12$$

$$\mathbf{w}(4) = \mathbf{w}(3) = \begin{bmatrix} 2 \\ 2 \\ 0 \end{bmatrix}$$

□ $k = 4, \mathbf{x}(4) = [1 \ 1 \ 1]^T \in \omega_2, \mathbf{w}(4) = [2 \ 2 \ 0]^T$

$$\mathbf{w}^T(4)\mathbf{x}(4) = [2 \ 2 \ 0] \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = 4$$

$$\mathbf{w}(5) = \mathbf{w}(4) - \alpha \mathbf{x}(4) = \begin{bmatrix} 2 \\ 2 \\ 0 \end{bmatrix} - (1) \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}$$



- If the training continues, the algorithm converges to the solution vector

$$\mathbf{w} = \mathbf{w}(12) = \begin{bmatrix} 1 \\ 1 \\ -3 \end{bmatrix}$$

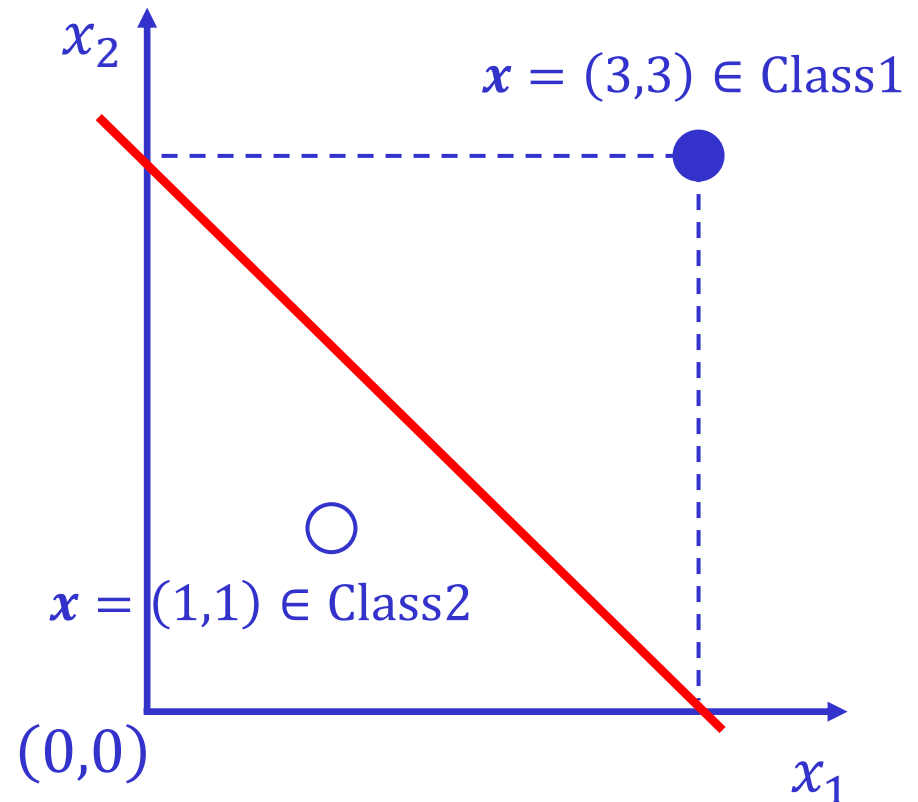
which gives the decision boundary

$$x_1 + x_2 - 3 = 0$$

Note:

$$y = mx + c$$

$$x_2 = -x_1 + 3$$

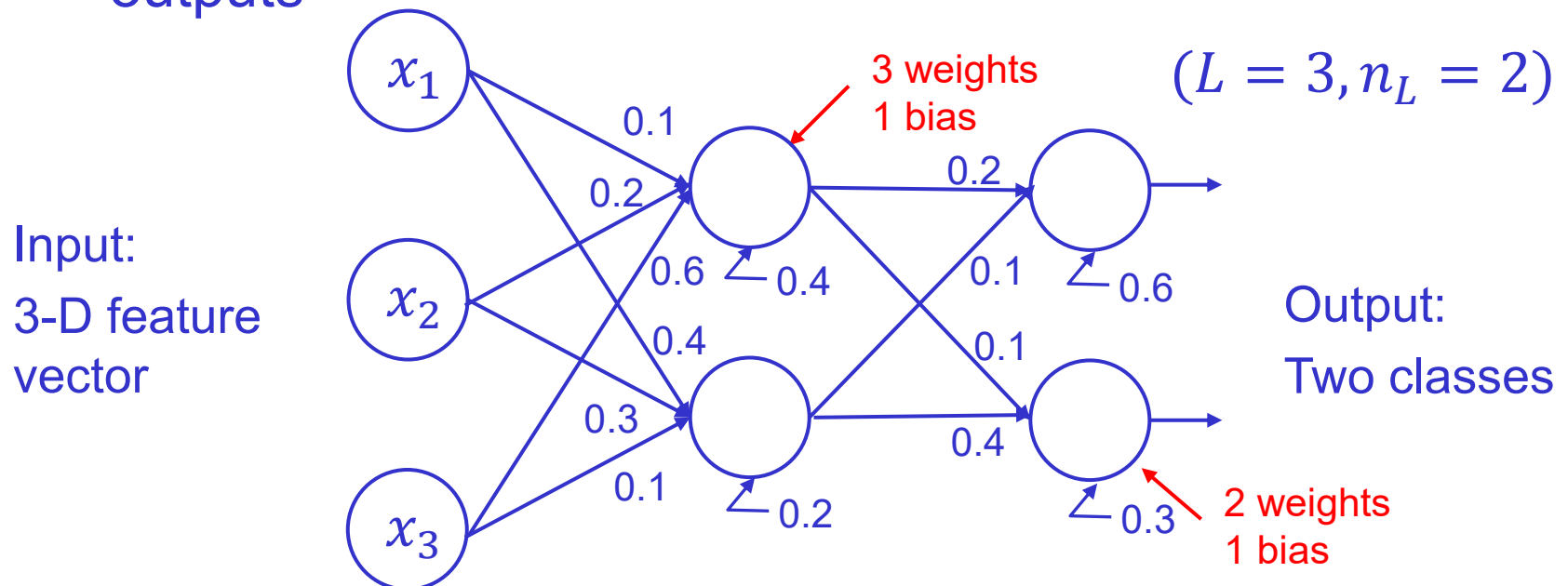




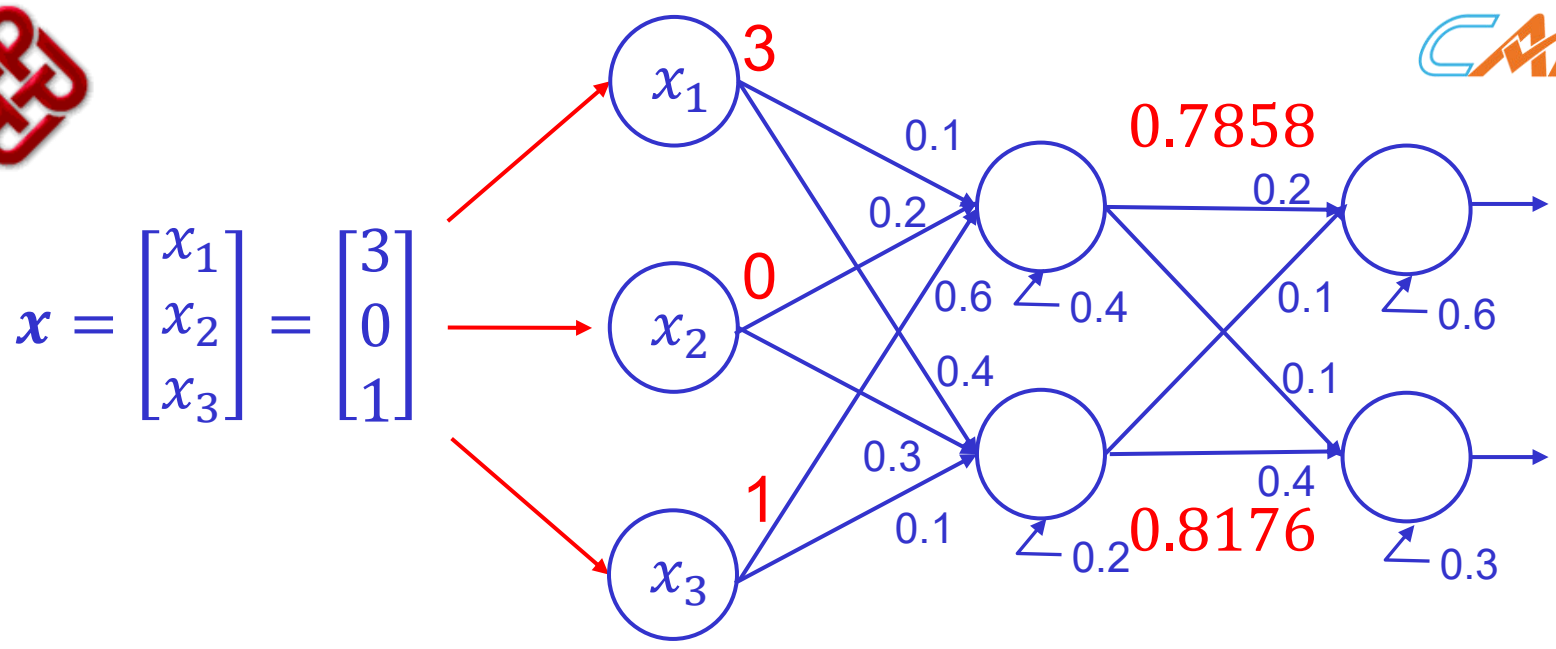
Example 2

A three-layer neural network

- The network consists of the input layer, one hidden layer, and the output layer
- The network accepts three inputs and has two outputs



Total number of parameters to be trained = $(3 + 1) \times 2 + (2 + 1) \times 2 = 14$

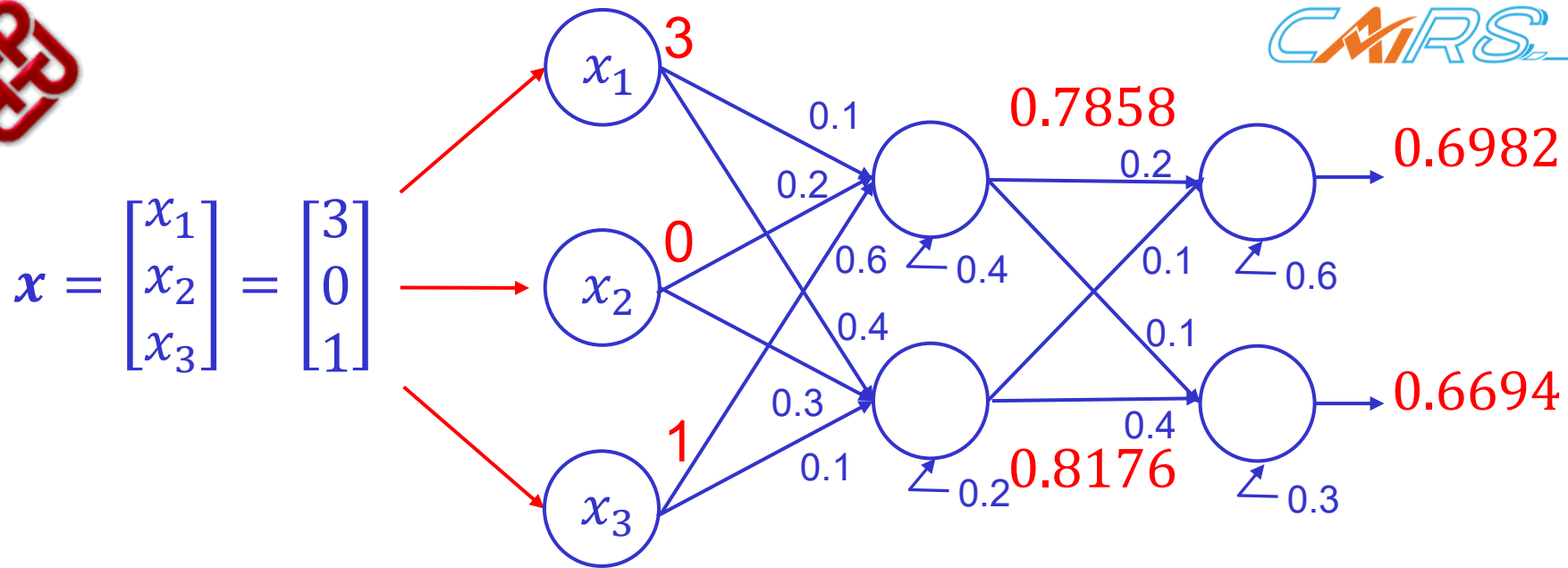


$$z_1(2) = \sum_{j=1}^3 w_{1j}(2)a_j(1) + b_1(2)$$
$$= (0.1)(3) + (0.2)(0) + (0.6)(1) + 0.4 = 1.3$$

$$a_1(2) = h(z_1(2)) = 1/(1 + e^{-1.3}) = 0.7858$$

$$z_2(2) = \sum_{j=1}^3 w_{2j}(2)a_j(1) + b_2(2)$$
$$= (0.4)(3) + (0.3)(0) + (0.1)(1) + 0.2 = 1.5$$

$$a_2(2) = h(z_2(2)) = 1/(1 + e^{-1.5}) = 0.8176$$



$$\begin{aligned} z_1(3) &= \sum_{j=1}^2 w_{1j}(3)a_j(2) + b_1(3) \\ &= (0.2)(0.7858) + (0.1)(0.8176) + 0.6 = 0.8389 \end{aligned}$$

$$a_1(3) = h(z_1(3)) = 1/(1 + e^{-0.8389}) = 0.6982$$

$$\begin{aligned} z_2(3) &= \sum_{j=1}^2 w_{2j}(3)a_j(2) + b_2(3) \\ &= (0.1)(0.7858) + (0.4)(0.8176) + 0.3 = 0.7056 \end{aligned}$$

$$a_2(3) = h(z_2(3)) = 1/(1 + e^{-0.7056}) = 0.6694$$



References

Youtube:

Learning from Data:

<https://youtube.com/playlist?list=PLnIDYuXHkit4LcWjDe0EwIE57WiGIBs08&si=M2ufojezUWtgdhns>

Introduction to Deep Learning:

<https://youtu.be/ErnWZxJovaM?si=ff9pKz4HGxIUx6tE>

Books:

Learning from Data - A short course:

[https://openeclass.panteion.gr/modules/document/file.php/PMS152/LEARNING/Abu-Mostafa%20Yaser%20S.%2C%20Malik%20Magdon-Ismail%2C%20Hsuan-Tien%20Lin%20%282012%29%20--%20Learning%20From%20Data %20A%20short%20course.pdf](https://openeclass.panteion.gr/modules/document/file.php/PMS152/LEARNING/Abu-Mostafa%20Yaser%20S.%2C%20Malik%20Magdon-Ismail%2C%20Hsuan-Tien%20Lin%20%282012%29%20--%20Learning%20From%20Data%20A%20short%20course.pdf)

The Little Book of Deep Learning: <https://fleuret.org/public/lbdl.pdf>

Dive into Deep Learning: <https://vi.d2l.ai/>